

Deep Reinforcement Learning for Real-Time Optimization of Pumps in Water Distribution Systems

Gergely Hajgató¹; György Paál, Ph.D., D.Sc.²; and Bálint Gyires-Tóth, Ph.D.³

Abstract: Real-time control of pumps can be an infeasible task in water distribution systems (WDSs) because the calculation to find the optimal pump speeds is resource intensive. The computational need cannot be lowered even with the capabilities of smart water networks when conventional optimization techniques are used. Deep reinforcement learning (DRL) is presented here as a controller of pumps in two WDSs. An agent based on a dueling deep q -network is trained to maintain the pump speeds based on instantaneous nodal pressure data. General optimization techniques (e.g., Nelder–Mead method, differential evolution) serve as baselines. The total efficiency achieved by the DRL agent compared to the best-performing baseline is above 0.98, whereas the speedup is around $2\times$ compared to that. The main contribution of the presented approach is that the agent can run the pumps in real time because it depends only on measurement data. If the WDS is replaced with a hydraulic simulation, the agent still outperforms conventional techniques in search speed. DOI: 10.1061/(ASCE)WR.1943-5452.0001287. © 2020 American Society of Civil Engineers.

Introduction

Ongoing efforts in the development of online measurement sensors and data acquisition (Nardo et al. 2018) let new techniques emerge in real-time control of water distribution systems (WDSs). The need for smart water networks and economic water supply is motivated mostly by the rapidly growing and urbanizing human population (United Nations 2018, 2019). Ever-growing water networks challenge operators in achieving cost-effective yet satisfactory operation. According to the comprehensive literature review of Mala-Jetmarova et al. (2018), operation optimization is the third most active topic among water supply-related applications.

Operation optimization is often treated as a pump scheduling problem, where dynamic programming (DP) methods are widely used because they are able to provide the global optimum if the number of variables is low. The iterative DP method (Zessler and Shamir 1989) is able to cope with the curse of dimensionality in large WDSs, but its application is restricted to convex objective functions. A more recent work by Bene (2013) introduced a method based on discrete dynamic programming that finds the optimal pump scheduling for medium-sized water networks. However, the multiple restrictions on the problem definition show that modern variants of dynamic programming are computationally still too expensive.

Savic et al. (2018) found that many state-of-the-art (SOTA) optimization techniques in WDS operation optimization involve some kind of metaheuristics. Ritter et al. (2020) presented a multiobjective

direct policy search technique where different heuristic approaches share the individuals of a population among different metaheuristics. This way, the different beneficial properties are combined from different metaheuristics, but incorrectly chosen hyperparameters of the distinct methods can alter the whole optimization process.

Moreover, these methods require a high amount of hydraulic evaluation, which makes the use of surrogate models a reasonable approach. Surrogate models are approximation models that are computationally cheaper than the corresponding—physically valid—numerical simulation. The input of a surrogate model is usually the same as for the simulation (e.g., boundary and initial conditions), but its output is restricted to just a handful of variables.

Surrogate models with high complexity (e.g., Gaussian process regression, artificial neural networks, k -nearest neighbors) are found to be misleading for nonconvex objective functions because these smooth the response surface, which leads to suboptimal solutions (Simpson et al. 2004). In concordance, Razavi et al. (2012) showed for water network simulations that substitutes with less complexity are more robust in cases where an unseen part of the explanatory variable space has to be discovered. Machell et al. (2010) introduced a hybrid approach where the governing equations of the hydraulic simulation are supported with an extended number of boundary conditions that reduce the computation cost of the hydraulic model. The additional information is extracted from online sensory data, including pressure and flow rate measures.

Optimization convergence can be enhanced by improving the optimum search itself to reduce the time of the overall process. A promising branch of these efforts is about finding the optimal parameters of the optimization algorithm. These parameters are called hyperparameters, and methods that optimize these hyperparameters during the single optimization process are referred to as hyperheuristic methods (Savic et al. 2018). McClymont et al. (2013) presented a Markov chain–based approximator that is superior in finding optimal hyperheuristics compared to Simple Random and TSroulWheel methods.

Though heuristics are robust in finding the global optima, the computational burden is still too high for real-life WDSs. Model predictive control (Pascual et al. 2011; Wang et al. 2017) overcomes the computational issue by using a two-layer optimization approach where the layers predict optimal on–off schedules for

¹Ph.D. Student, Dept. of Hydrodynamic Systems, Budapest Univ. of Technology and Economics, Budapest 1111, Hungary (corresponding author). ORCID: <https://orcid.org/0000-0003-4283-126X>. Email: ghajgato@hds.bme.hu

²Professor, Dept. of Hydrodynamic Systems, Budapest Univ. of Technology and Economics, Budapest 1111, Hungary. Email: gypaal@hds.bme.hu

³Associate Professor, Dept. of Telecommunications and Media Informatics, Budapest Univ. of Technology and Economics, Budapest 1117, Hungary. Email: toth.b@tmit.bme.hu

Note. This manuscript was submitted on October 17, 2019; approved on May 26, 2020; published online on August 25, 2020. Discussion period open until January 25, 2021; separate discussions must be submitted for individual papers. This paper is part of the *Journal of Water Resources Planning and Management*, © ASCE, ISSN 0733-9496.

pumps and valves in different time horizons. The method is able to run large WDSs in real time, and the capabilities are presented on the example of the water network of Barcelona. Iterative extended lexicographic programming (iELGP), introduced by Abdallah and Kapelan (2017, 2019), copes even with continuous decision variables like the set-point of variable-speed pumps (VSPs) and chlorine dosing machines. With iELGP, Abdallah and Kapelan (2019) achieved SOTA performance on the C-Town benchmark problem wherein both fixed-speed and variable-speed pumps were presented.

Water distribution system operation also benefits from recent advances in machine learning. Artificial neural networks are used in a wide variety of applications, from demand forecast (Guo et al. 2018) through attack detection (Taormina and Galelli 2018) to pump scheduling (Bhattacharya et al. 2003).

Bhattacharya et al. (2003) used an artificial neural network (ANN)-based reinforcement learning agent as an optimal controller for switching on or off pumps in a water network. The input of the ANN is water levels in reservoirs besides the water demand in the WDS, whereas its output is the decision to operate a pump or not. The neural network is trained with reinforcement learning (RL) in a manner such that the RL agent is encouraged to replicate the moves of the master controller algorithm.

Mahootchi et al. (2007) introduced an agent based on reinforcement learning for the control of one reservoir in a water distribution network and found that the RL approach achieved comparable results to stochastic dynamic programming in the benchmark case.

A sophisticated optimization technique was introduced by Castelletti et al. (2013) for multiobjective tasks, where the importance of the standalone objectives is unknown or uncertain at the beginning of the optimization. Recombining these objectives yielded a different optimal solution, hence a movement on the Pareto frontier. Castelletti et al. (2013) approximated the optimal policy with fitted q -iteration (FQI) during an optimization run but also added the weights of the standalone objectives to the state space of the environment used in FQI. The approximation of the optimal policy was constructed as the function of the importance weights, and the frontier can be drawn from one single run with sampling the weight space as well.

All the pump scheduling methods previously rely on the simulation of the water distribution system that limits their usability when the size or the complexity of the water network is too large. The most resource-intensive method is finding optimal pump speeds for a time step because the descriptive variables are continuous, and the evaluation of every trial needs to run the hydraulic simulation as well.

A reinforcement learning agent for real-time control for variable-speed pumps is presented in this paper. The pump scheduling problem is omitted and only the optimum speed search for a given demand-distribution is realized in a smart water network. A significant novelty of the method is that the hydraulic simulation of the WDS is not needed for the controller because it relies only on measurement data. Moreover, the training is carried out in such a way that the agent is able to develop different policies compared to the reference, so that the agent can outperform the master algorithm it was trained with. If online pressure data are not available, the hydraulic simulation of the water distribution system can be used as a surrogate of the real WDS, like in the case of existing optimization methods. In such cases, the agent-based control is able to outperform current optimization techniques in the number of hydraulic model evaluations. Because the proposed technique is able to cope with multiple objectives with a simple weighted sum of objectives, it can serve as the optimal pump speed search algorithm in hybrid pump schedule optimization solutions.

The paper is organized as follows. First, the simplified problem is introduced including the reference methods. The details of reinforcement learning methodology are described thereafter. Then, the settings of the case study, including the water distribution networks and the training settings, are presented. The performance of the agent is evaluated on the WDSs and conclusions about the training are drawn. Finally, the key findings and possible applications are given.

Methodology for Pump Speed Control with Reinforcement Learning

Problem Definition and Reference Solutions

The present study aims at providing optimal set-points for variable-speed pumps in a WDS according to given nodal pressures. Hence, the conventional pump scheduling problem is simplified and the problem is formulated as follows.

Instantaneous operation point optimization is carried out, neglecting temporal changes in the demand. The competing algorithms have to find the optimal pump speeds in terms of hydraulic efficiency with respect to randomly generated demands. Also, the inflow and outflow into and from the tanks should be minimized, and pressure (head) should be kept in a specific range in all of the nodes. Customer satisfaction implies a lower pressure limit, whereas the mechanical strength of water fittings and other elements of the water network imply an upper limit as well.

These objectives are formulated into a simple weighted sum of objectives. The meaning of this combined value is twofold because it serves as an objective value for the widespread optimization algorithms and is interpreted as the value of the state in the Markovian decision process described later. The rating of the final state is the same, regardless of whether it is achieved by the reference optimization methods or the reinforcement learning-based agent.

The calculation depends on the observation of the environment that is the ensemble of the state variables and some other quantities that are not involved in the decision-making but are measurable. The algorithm of the calculation is presented in Algorithm 1. The input to the algorithm is the state variables S , the set of the pump efficiencies E_{pump} , the volumetric flow rate F_{pump} delivered by the pumps, and the signed volumetric flow rate F_{tank} of the tanks.

Algorithm 1. Calculating the state value $v(S)$

```

1: function ComputeStateValue( $S, E_{\text{pump}}, F_{\text{pump}}, F_{\text{tank}}$ )
2:    $H = \{h_1, \dots, h_N\} \leftarrow \text{ExtractNodalHeads}(S)$ 
3:    $n_{\text{wrong}} \leftarrow 0$ 
4:   for all  $h \in H$  do
5:     if  $h < h_{\min}$  or  $h > h_{\max}$  then
6:        $n_{\text{wrong}} \leftarrow n_{\text{wrong}} + 1$ 
7:     end if
8:   end for
9:    $c_{\text{tot}} \leftarrow \sum F_{\text{pump}} + \sum F_{\text{tank}}$ 
10:   $\eta_{\text{tot}} \leftarrow \prod E_{\text{pump}}$ 
11:   $r_{\text{feed}} \leftarrow \frac{c_{\text{tot}}}{c_{\text{tot}} + \sum_{i=1}^{\#F_{\text{tank}}} |f_i|}$ 
12:   $r_{\text{satisfaction}} \leftarrow 1 - \frac{n_{\text{wrong}}}{\#H}$ 
13:   $r_{\text{eff}} \leftarrow \frac{\eta_{\text{tot}}}{\eta_{\text{limit}}}$ 
14:   $v \leftarrow w_{\text{satisfaction}} * r_{\text{satisfaction}} + w_{\text{eff}} * r_{\text{eff}} + w_{\text{feed}} * r_{\text{feed}}$ 
15:  return  $v$ 
16: end function

```

The state value v is a weighted combination of three objectives:

1. The value $r_{\text{satisfaction}}$ represents the satisfaction of consumers that is characterized by the number of the insufficiently supplied nodes n_{wrong} wherein the head is out of the acceptable range bounded by h_{min} and h_{max} . The number of problematic nodes is divided by the number of all nodes in the water distribution system, and the ratio is subtracted from 1. The outcome is the basis of the first weighted objective, the satisfaction ratio.
2. The value r_{eff} represents the efficiency of the pumps, and it is just the product of the standalone pumps feeding the network divided by the product of the theoretical peak efficiencies of all of the pumps.
3. The value r_{feed} represents the feed ratio of the water network that is solved by the pumps supplying water from the wells or other sources on the one hand and by the tanks and reservoirs on the other hand.

In the current experiment, using the water reserve is considered expensive, and using the reserve is punished by a lower objective value, which is a simplification of the problem. However, this methodology is easily changeable by adding the target inflows and outflows to the state value and comparing the actual ones to those. In this way, the agent would be forced to learn to maintain the water flux of the tanks, but this is a harder problem to solve. For the time being, this is ignored by forcing the agent to minimize the water flux of the tanks. Thus, the feed ratio is the amount of total consumption divided by the sum of the total consumption and the water flux of the tanks.

The standalone objectives are also presented in Eq. (1), where the notations are the same as for Algorithm 1 except for f_{tank} , which denotes the sum of the absolute value of inflow and outflow for every tank

$$\begin{aligned} r_{\text{satisfaction}} &= 1 - \frac{n_{\text{wrong}}}{n_{\text{nodes}}}, \\ r_{\text{eff}} &= \frac{\eta_{\text{tot}}}{\eta_{\text{limit}}}, \\ r_{\text{feed}} &= \frac{c_{\text{tot}}}{c_{\text{tot}} + f_{\text{tot,tank}}} \end{aligned} \quad (1)$$

The previous formulation of the standalone objectives leads to three measures, whose maximum values are all 1.0. These are then weighted separately by constant factors. The coefficients in the presented experiments are chosen arbitrarily as 8/16, 5/16, and 3/16 for the ratio of the satisfaction, efficiency, and feed, respectively.

The task was solved with different optimization techniques to provide reference: fixed step-size random search (FSSRS), particle swarm optimization (PSO), differential evolution (DE), and the

Nelder–Mead method. PSO, DE, and the Nelder–Mead method are able to solve operation point optimization of VSPs; see the work of Baltar and Fontane (2008), Odan et al. (2015), and Celi et al. (2017), respectively.

A suboptimal optimization method that is referred to as one-shot random trial (one-shot RT) was also applied on the task. This method sets pump speeds randomly inside the feasibility domain, and it serves as a baseline: the reinforcement learning agent has to be able to outperform this approach if it can interpret any information on the connection of nodal pressures and optimal pump speed settings.

Dueling Deep Q-Network Agent as a Real-Time Controller for Water Networks

The drawback of the reference methods is the incapability to accomplish real-time control because of the necessity of computationally expensive simulation of the hydraulics. In this paper, a dueling deep q -network agent is introduced to optimize the speeds of the pumps in a water distribution system. The agent relies on simulation data in the training phase only; thus, the controlling is feasible in real time, regardless of the size of the water network and the number of pumps controlled.

The proposed technique is based on Markov decision processes (MDPs) that are key building blocks of reinforcement learning. In the terminology of MDPs, an agent works in an environment by taking actions according to a policy. The environment is observable to the agent through its state that changes according to the actions. The instantaneous quality of a state is measured by the reward that is signaled to the agent. Rewards of a decision sequence are combined in the value function. The value function measures the overall quality of the starting state in the decision sequence according to the given policy. The goal of the learning is to find a policy with which the agent can govern the environment to the most advantageous state from any initial state.

In the present study, the agent is a dueling deep q -network, and it interacts with the environment that is based on the water network. The decision-making sequence and most of its concepts are depicted in Fig. 1 [originating from Sutton and Barto (2018)], where t and $t + 1$ refer to two consecutive steps (also referred to as time steps) in the sequence.

According to Fig. 1, the communication between the environment and the agent during the training is as follows:

1. The environment provides its state variables—the ensemble of the nodal pressures and the pump speed ratios—to the agent in a concatenated vector. Speed ratios are the actual pump speed divided by the nominal pump speed.
2. The agent maps the state values to an action according to its current policy and executes it in the environment. The action can be the increase or decrease of one of the pump speed ratios or keeping the speed ratios as they were set in the previous step. The latter is referred to as siesta.
3. The new hydraulic properties are provided by the hydraulic simulator, and the value of the new state is calculated. Also, the reward is calculated from the state value and some other properties of the environment.
4. The environment propagates the new state and the corresponding reward to the agent to induce the next move.
5. Finally, (1) the preceding state, (2) the preceding action taken, (3) the corresponding reward, and (4) the new state are stored as a training sample with which the action-value estimator is updated later.

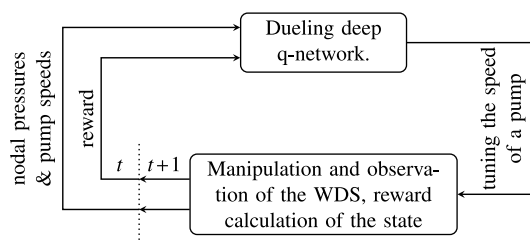


Fig. 1. Agent–environment ecosystem tailored to the controlling of the pumps in a water distribution system.

This cyclic process goes on until the environment reaches a terminal state, which can be caused by two distinct events. On the one hand, the agent is allowed to run until a fixed number of steps to get to the most advantageous state. The environment keeps track of the number of steps taken, and it terminates the episode when the agent reaches the limit. On the other hand, the agent can decide to do nothing instead of changing one of the speed ratios. If the agent adheres to a state for a predefined number of consecutive steps, the episode is terminated as well.

Seeking to achieve the optimal policy leads to the grading of different policies. A policy is regarded as better than another if the value function is equal or higher for all states by taking actions under that one. The value function can be formulated to depend on different variables; the related form is the action-value function, that is for policy π

$$q_{\pi}(s, a) \doteq \mathbb{E}_{\pi} \left(\sum_{k=t+1}^T \gamma^{k-t-1} R_k | S_t = s, A_t = a \right) \quad (2)$$

The value is represented as an expected value of the sum of rewards R_k in the forthcoming sequence started from state S_t and taking action A_t under policy π . The importance of future rewards can be attenuated with the discount factor γ , such that $\gamma \in (0, 1]$.

If the agent runs under the optimal policy, its actions maximize the reward in every state transition; thus, the optimum of the action-value function is

$$q_{\pi^*}(s, a) \doteq \mathbb{E}_{\pi^*} [R + \gamma \max_{a'} q^*(s', a') | S_t = s, A_t = a] \quad (3)$$

where the forthcoming action and state are denoted by a prime; and q^* refers to action values under the optimal policy.

Eq. (3) can be solved in different ways; for the relevant techniques, see Sutton and Barto (2018). SOTA solutions use artificial neural networks as value function estimators to cope with the complexity of the equation. ANNs are good candidates for the task because they are universal function approximators, as shown by Cybenko (1989).

A variant of deep q -networks [DQNs, introduced by Mnih et al. (2013)] is chosen in the present study to approximate the action-value function. The term deep refers here to the fact that the neural network architecture has two or more hidden layers. The agent can be trained by minimizing the loss

$$L_i(\theta_i) = \mathbb{E}_{\pi} \{ [r + \gamma \max_{a'} q(s', a', \theta_{i-1}) - q(s, a, \theta_i)]^2 \} \quad (4)$$

where θ_i corresponds to the weights in the i th update of the deep q -network (a deep neural network in this case); and L_i = mean squared error defined between the $(i-1)$ th and i th approximation of the value function. A significant advantage of Eq. (4) over Eq. (3) is that it does not have to evaluate a whole decision sequence to get a new assumption on the value of the updated state; hence, it can be trained off-policy.

Eq. (4) is optimized with a stochastic gradient descent method because it is a general, yet effective, optimization algorithm for training ANNs (Bottou and Bousquet 2008). Moreover, the last layer of the deep q -network is forked: one layer approximates the value of the current state, whereas the other layer estimates the advantage in the value by taking the action. This is intended to speed up training convergence, and the technique is called dueling deep q -networks, as introduced by Wang et al. (2015).

A crucial component of the reinforcement learning setting is the rewarding of the agent because it influences the behavior of the trained agent. Algorithm 2 summarizes the step procedure of the rewarding system of the present study.

Algorithm 2. Step procedure of the agent

```

1: procedure Step( $a$ )
2:    $n_{siesta} \leftarrow 0$ 
3:    $n_{steps} \leftarrow n_{steps} + 1$ 
4:    $terminal \leftarrow n_{steps} = N$ 
5:    $r \leftarrow penalty$ 
6:   if  $a \neq siesta$  then
7:      $PS \leftarrow \text{UpdatePumpSpeeds}(a)$ 
8:      $\delta \leftarrow \|PS - PS_{ref}\|$ 
9:     if  $any(PS < ps_{lo})$  or  $any(PS > ps_{hi})$  then
10:       $PS \leftarrow \text{RevertAction}(a)$ 
11:   else
12:     if  $\delta < \delta^*$  then
13:        $r \leftarrow \delta \cdot constant$ 
14:     end if
15:      $S \leftarrow \text{UpdateState}(a)$ 
16:      $E_{pump}, F_{pump}, F_{tank} \leftarrow \text{Observe}(S)$ 
17:      $v \leftarrow \text{ComputeStateValue}(S, E_{pump}, F_{pump}, F_{tank})$ 
18:      $\delta^* \leftarrow \delta$ 
19:   end if
20: else
21:    $n_{siesta} \leftarrow n_{siesta} + 1$ 
22:   if  $n_{siesta} < 3$  then
23:     if  $1 - v/v_{ref} < \epsilon$  then
24:        $r \leftarrow n_{siesta} \cdot constant$ 
25:     end if
26:   else
27:     if  $1 - v/v_{ref} < \epsilon$  then
28:        $r \leftarrow bonus$ 
29:     end if
30:      $terminal \leftarrow \text{True}$ 
31:   end if
32: end if
33: return  $r, S, terminal$ 
34: end procedure

```

The reward is based upon the state value, but it is not a direct match because other goals have to be reached beside optimality, as shown in Algorithm 1. Because the control is realized stepwise, the agent has to be taught to set the optimal speeds in as few steps as possible. The basic idea behind the presented step function, including the reward calculation, is as follows. At the beginning of every new scenario; that is, after the generation of a new demand distribution, the optimal setting of the pump speeds is obtained by the Nelder–Mead method. This speed setting PS_{ref} and the corresponding state-value v_{ref} is considered as a reference, guiding the agent softly in the search space. After the agent intervenes, the reward is based upon the closeness δ of the reference solution and the agent's solution in the search space, defined by the feasible pump speed ratios. At this stage, the agent gets a reward r if it gets closer to the reference solution on a monotonous trajectory; that is, it does not make steps increasing the distance δ between the solutions. The agent can also choose to make a siesta; in this case, the reward is computed differently. In siestas, the actual value of the state is computed according to Algorithm 1 and compared to the reference value. If the current state value is higher than the 98% percent of the reference value (i.e., the value of $1 - v/v_{ref}$ is at most ϵ), the agent is rewarded with a slightly higher score. The immediate reward achieved in the first siesta is doubled if the agent chooses to stay in this beneficial state for the second time. Finally, the episode is terminated after the third consecutive siesta, and the agent gets an extra reward that is significantly higher than the rewards that are achievable with the active moves. In contrast, the agent gets a negative reward whenever it recedes from the reference speed setting

with an active action or if it stays in a suboptimal position, that is, if it makes a siesta in a state with a value below 98% of the reference. A penalty is given even when the agent tries to set pump speeds lower or higher than the feasibility limits ps_{lo} and ps_{hi} , respectively. With this concept, the agent is incited to move along the optimal position continuously and terminate the episode there. The 2% tolerance in the comparison of the state values is needed because of the different variable space from where the solution is acquired with the reference method and reinforcement learning agent: the Nelder–Mead method chooses candidate solutions from a continuous variable space, whereas the dueling deep q -network handles a discrete action space; thus, only a restricted set of pump speeds is available compared to Nelder–Mead. Although the discretization can be arbitrarily fine, the dimensionality of the q -function—and hence the length of the training of its estimator—depends on the size of the discrete parameter space.

The soft guidance means in the previous process that the agent can achieve high rewards even far from the reference solution if it has a similar value. This is encouraged during training by taking random actions in a part of the whole process, thanks to the ϵ -greedy policy.

After training, the agent relies only on the observation of the environment to take its next action; thus, the calculation of the reward—including the computationally expensive reference calculation—is completely omitted.

Case Study

Reference Water Distribution Systems

The proposed controller is trained and tested on the modified Anytown and D-Town (referred to as C-Town in some literature) water distribution systems that are common among the WDS design and management research community. These networks serve as benchmarks for water network design methods with which the networks can be modified and extended to comply with increased water needs and expanding consumption areas. The original topology of both WDSs is accessible online from the web page of the Centre of Water Systems research group at the University of Exeter (1986). Moreover, the numerical models of the WDSs are also available in an EPANET-compatible format. The modified versions are referred to as Anytown-mod and D-Town-mod in the following.

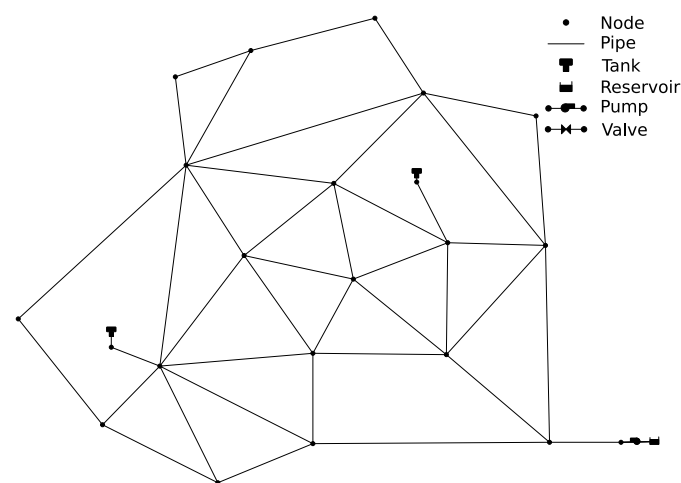


Fig. 2. Topology of the Anytown water distribution system. Network elements plotted: nodes, pipes, pumps, and tanks.

The topology of the networks is depicted in Figs. 2 and 3 for the Anytown and D-Town test cases, which are the same for Anytown-mod and D-Town-mod, respectively. The networks are slightly modified, as described subsequently. The primary purpose of Anytown-mod is to provide a simple, easily solvable problem that serves as a proof of concept: it has a global optimum that can be found with conventional methods easily. If the agent performs poorly on Anytown-mod, then there is no chance that it can solve more complex tasks. D-Town-mod is a more challenging problem whose solution is not trivial because of the multidimensional parameter space of the optimization loss function. If the agent performs well on D-Town-mod, then the RL-based agent with the proposed reward function is a good candidate to serve as a real-time controller in real water networks. The main properties of the two water distribution systems are listed in Table 1. Both water networks share their topology with the original version published by the University of Exeter (1986); the differences are summarized subsequently.

Anytown-mod is almost identical to Anytown because only the number of pumps in the single pumping station is reduced to two from three, and both of the pumps operate with the same continuously variable speed. The performance curves of the pumps are the same as in the original network.

In the case of D-Town-mod, the control logic is removed, and all pumps operate in the water distribution network. The performance curves are kept for the head, and fictitious curves are added for the efficiency.

An environment is created that serves as an interface between the agent and the water distribution system. The purposes of the environment are the interpretation of the commands of the agent, reward calculation, and reinitialization of the water network. The state of the

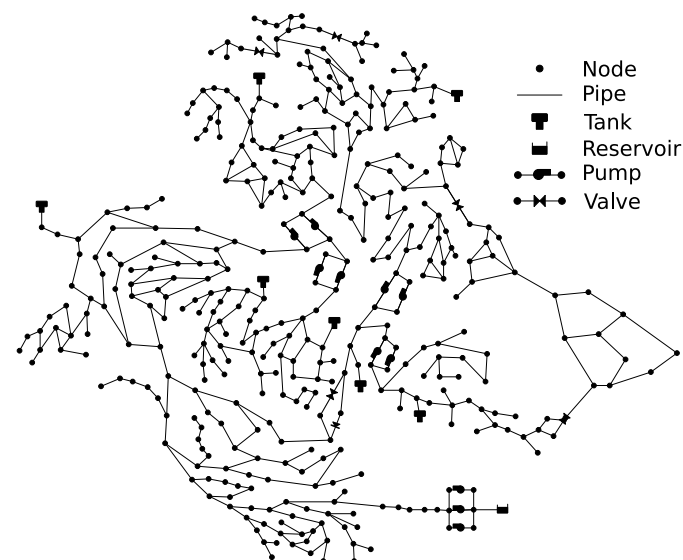


Fig. 3. Topology of the D-Town water distribution system. Network elements plotted: nodes, pipes, pumps, and tanks.

Table 1. Main properties of the water distribution systems

Attribute of the water network	Anytown-mod	D-Town-mod
Number of nodes	22	399
Number of pipes	41	443
Number of pump stations	1	5

water network can be initialized by randomizing the demand map and by randomizing the pump speeds or resetting them to the original value. The original demands in the task definition of the water network are stored, and the total sum of the original nodal consumption is calculated and stored at the beginning of training.

Demand variations are replaced with randomized demand distributions over the water distribution system. This can be done because of training the agent with an off-policy method. It is not necessary to show real-life demand variations to the agent in the training phase because it is trained off-policy.

The randomization of the nodal consumption is done in three steps to be able to mimic the total demand variation during day and night and the nodal demand variation as well. First, new demand is generated in every node as the product of the original demand and a random value from a truncated normal distribution. Second, new total demand is generated as the product of the original total demand and a random value from a uniform distribution. Third, the nodal consumptions are rescaled such that the sum is equal to the one generated in the second step. The process is summarized in Algorithm 3 where the set of original nodal consumptions and randomized nodal consumptions are denoted by C_0 and C , respectively.

Algorithm 3. Initializing nodal consumptions

```

1: function InitNodalConsumption( $C_0$ )
2:    $C \leftarrow C_0$ 
3:    $c_{tot} \leftarrow \text{RndUniform}(\text{lo} = 0.3, \text{hi} = 1.1) * \sum C_0$ 
4:    $c_{tot*} \leftarrow 0$ 
5:   for all  $c \in C$  do
6:      $c \leftarrow \text{RndTruncNormal}(\text{mean} = 1.0, \text{stddev} = 1.0, \text{lo} = 0.7, \text{hi} = 1.3) * c$ 
7:      $c_{tot*} \leftarrow c_{tot*} + c$ 
8:   end for
9:   for all  $c \in C$  do
10:     $c \leftarrow c * \frac{c_{tot}}{c_{tot*}}$ 
11:   end for
12:   return  $C$ 
13: end function

```

This randomization method allows control of the demand map both in relative and absolute terms. The nodal consumption randomized one by one mimics the relative variation of the demand map. Meanwhile, the overall change in consumption through the day–night cycle is imitated by the variation of the total demand; that is, the total demand of the area covered by the water distribution network is considered first; then, the total value is partitioned among the nodes.

Both distributions are chosen arbitrarily, and their probability density functions are depicted in Fig. 4. The lower and upper bounds of the uniform distribution are 0.3 and 1.1, respectively, whereas the normal distribution has a mean 1.0 and standard deviation 1.0 with tails cut off below 0.7 and above 1.3. The shape of the truncated normal distribution is similar to the uniform distribution with this parameter set, although multipliers close to 1 have slightly higher probabilities during demand randomization than remote values.

The pumps feeding the water network are considered variable-speed pumps driven by, for example, frequency converters, in case of both water networks. The absolute pump speed is not considered; the ratio compared to the nominal pump speed is recorded instead. In the current environment, the speeds can be lowered to half of the maximum speed because water transportation is infeasible at lower speeds due to insufficient pump heads relative to the

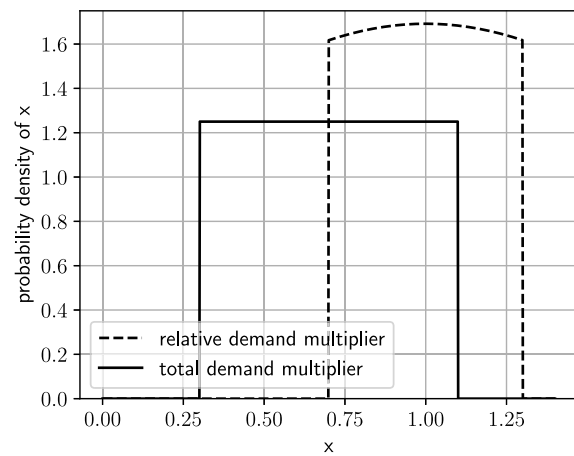


Fig. 4. Probability distributions for the three-stage demand randomization.

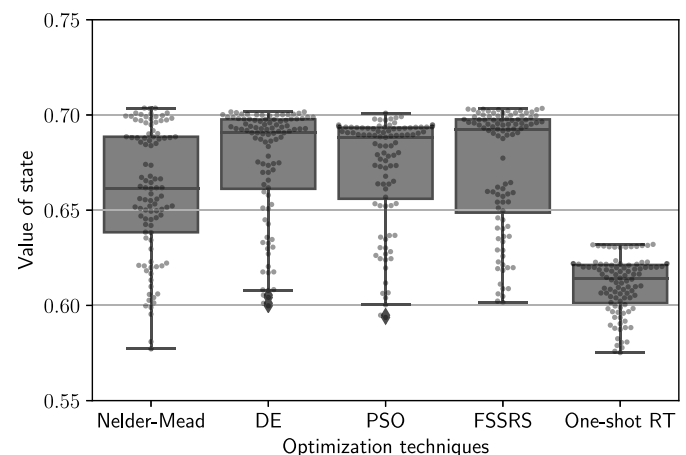


Fig. 5. Results of the optimization on the test set with different techniques. The higher state values are the better.

geodetic height of the nodes. The cause of the lower speed limit is valid for both WDSs.

Finally, the well from where the pumps feed the network is considered a reservoir with a water surface large enough to ensure a stationary water level during the speed adjustment of the pumps.

The presented results are obtained with the default hyperparameters of the optimization algorithms. In Fig. 5, the average performance of the different algorithms is presented on the test set in the case of Anytown-mod and D-Town-mod. Moreover, the box plot of the difference calculated for every test scenario is depicted in Fig. 6, where the results achieved by the differential evolution are chosen as a reference.

The similarity of the results suggests that each technique is able to find the empirical global optima, except for the one-shot random trial, which is reasonable.

The single pump station in Anytown-mod means that the pump speed optimization problem has a one-dimensional parameter space; hence, evolutionary algorithms and related techniques are inefficient in finding the solution because crossover-like operations are meaningless on one-dimensional candidate solutions. Thus, only the performance achieved by the Nelder–Mead method was considered in the evaluation of scenarios from the Anytown-mod case.

Training Procedure of the Agent

Two separate agents were trained during the experiment: one to solve the Anytown-mod problem and one to solve the D-Town-mod problem.

The agents were trained on the water networks separately; thus, during a training session, only one of the water distribution systems is used in the environment, and the trained agent will perform well only on the specific water network that was used for training. The agents trained with different hyperparameters; the common part is just the underlying algorithm and the fact that a densely connected neural network is used as an action-value estimator.

The training will be described next. At the beginning of every episode, the demands were randomized according to Algorithm 3; the pump speed ratios were randomized according to a uniform distribution between the feasibility limits. After every 1/25th of the total number of training steps, the training was suspended, and the performance of the agent was tested on 100 different scenarios. These scenarios were generated before the training, and they served as validation data. An additional 50 test scenarios were generated; these scenarios were solved by the agent only after the training was completed.

A hyperparameter optimization was carried out prior to the training with a tree-structured parzen estimator (TPE) (Bergstra et al. 2011). The critical hyperparameters were identified according to the previous experience of the authors. Table 2 summarizes possible hyperparameter values from where the optimal values were selected by the TPE. Additional information on the sensitivity on the parameters and hyperparameter optimization is not presented due to length considerations.

The input layers of the neural networks had exactly as many neurons as nodes in the water network because the nodal pressures divided by the highest shut-off head of the pumps are handed over to the action-value estimator. The activation functions were rectified linear units (ReLU) that were introduced in Nair and Hinton (2010).

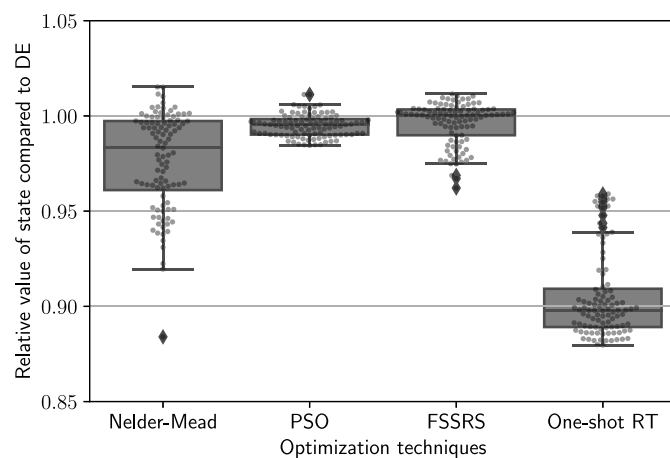


Fig. 6. Box and swarm plots of the performance of the optimization algorithms compared to the differential evolution method. A value of 1.00 means the same performance that is achieved with DE.

Table 2. Possible hyperparameters for training

Hyperparameter	Anytown-mod	D-Town-mod
Learning rate	$\{10^{-4}, 10^{-3}, 10^{-2}\}$	
Discount-factor	$\{0.9, 0.99, 0.999, 0.9999\}$	
Batch size	$\{8, 16, 32, 64\}$	$\{16, 32, 64, 128\}$
Number of neurons in the hidden layers	$\{[24], [24, 6], [48, 12], [24, 12, 6], [48, 32, 12]\}$	$\{[300], [300, 10], [128, 12], [256, 32, 12], [256, 128, 12]\}$

ReLU activation can handle input data outside the normalized range when the data are positive valued, as is the case with nodal pressures under normal circumstances. Nodal pressures were handed over to the neural network in vectors in every scenario; that is, the k th element of the pressure vector belonged to the same node under all circumstances. The training was carried out with the ϵ -greedy policy starting with an exploration factor of 0.95 and linearly decreasing to 0.0 at the final episode. The ϵ -greedy policy means that the agent makes random moves with a probability of the exploration factor, that is, acting off-policy. This behavior ensures the possibility of finding better policies than the reference one; that is, the agent is able to achieve high rewards even without mimicking the reference strategy. The discount factor affects the anticipatory behavior of the agent: the higher the value, the more prescient the agent is. A scenario was 40 and 200 timesteps long in the case of Anytown-mod and D-Town-mod, respectively. Moreover, the agents made moves respectively 50,000 and 1,000,000 times in the smaller and the larger water distribution network before the final test. The state-action-reward-new state data were saved to a so-called replay memory from where the training algorithm takes samples to fill the training batch for every update of the action-value estimator. The hyperparameters of both agents are summarized in Table 3.

The hydraulic backend system was ensured by the numerical model of the water distribution systems, and these models were evaluated in EPANET, which is a popular open-source hydraulic simulator introduced by Rossman (1993). Because the main parts of the computer code are written in Python version 3.7, only the core hydraulic calculations were computed by EPANET; modification of the problem definition and querying the results were conducted by the EPYNET version 1.1 interface, which is a Python wrapper for EPANET. EPYNET was developed by IT experts at the Dutch drinking water company Vitens and is publicly available by Heinsbroek (2016). The dueling deep q -network agent is a prepackaged one from the reinforcement learning library Stable Baselines version 2.10.0 (Hill et al. 2018). The authors have developed the environment with which the agent interacts in Python, and its interface is compatible with OpenAI gym version 0.17.2 for the seamless pairing of prepackaged agents with the same standard interface. Hyperparameter optimization was carried out with the Optuna version 1.4.0 package (Akiba et al. 2019) for Python.

Table 3. Hyperparameters of the agents

Hyperparameter	Anytown-mod	D-Town-mod
Learning rate	0.0001	
Discount-factor	0.99	0.9
Batch size	8	64
Number of neurons in hidden layers	[48,32,12]	[256,128,12]
Size of replay memory	25,000	350,000
Type of activation function	Rectified linear unit	
Initial exploration factor	0.95	
Final exploration factor	0.00	
Maximum episode length	40	200
Speed ratio increment	0.05	
Number of initialization steps	1,000	10,000

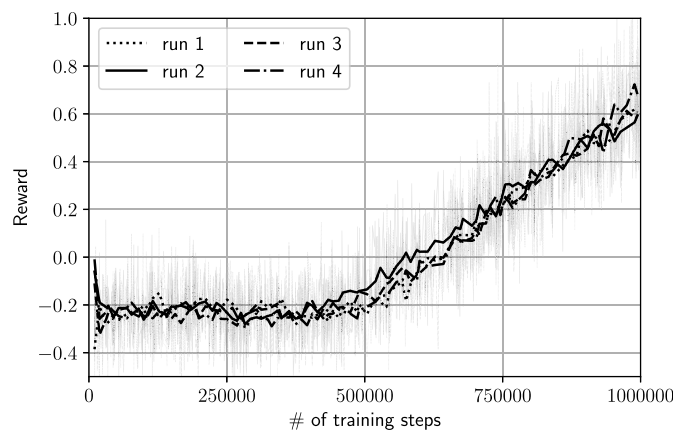


Fig. 7. Reward during the training at D-Town-mod.

Evaluation and Results

The history of the rewards and episode rewards is presented first for the D-Town-mod case in Figs. 7 and 8, respectively. The history of the rewards and the episode rewards were similar in the Anytown-mod case; thus, those are not presented in the paper due to length considerations. Because the parameters of the action-value estimator were initialized randomly, the agent had no prior knowledge of the problem. However, some initialization steps were taken by the agent to collect at least as many training samples with which the replay memory can be filled. The underlying model—the artificial neural network—was updated first only after the initialization steps; thus, the training history is not available for the preceding steps.

Because of the random initialization, different training sessions can produce different results. Moreover, the exploration capability of the agent introduced some stochastic behavior in the test performance of the agent through training. Thus, four training sessions are presented for each problem, and the results were smoothed by exponential moving averaging at $\alpha = 0.3$. As the agent started taking steps according to a random policy, the achieved rewards were low at the beginning of the training. The randomness of acting was maintained with a decreasing magnitude as the training went on; thus, the rewards should increase as the action-value estimator is updated with more and more training samples. Both the immediate and episode rewards saturated at the end of the training indicating

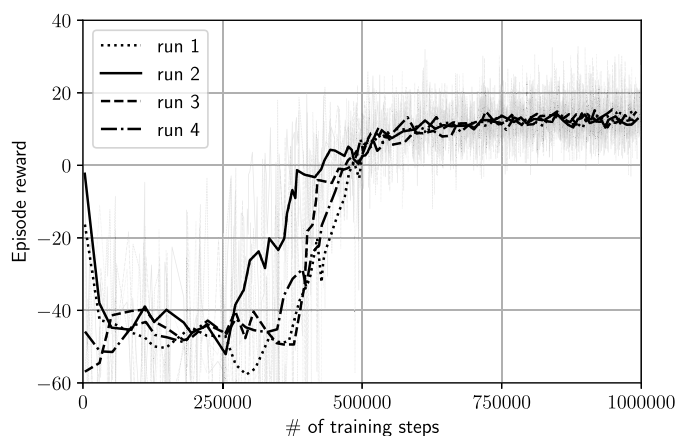


Fig. 8. Episode reward during the training at D-Town-mod.

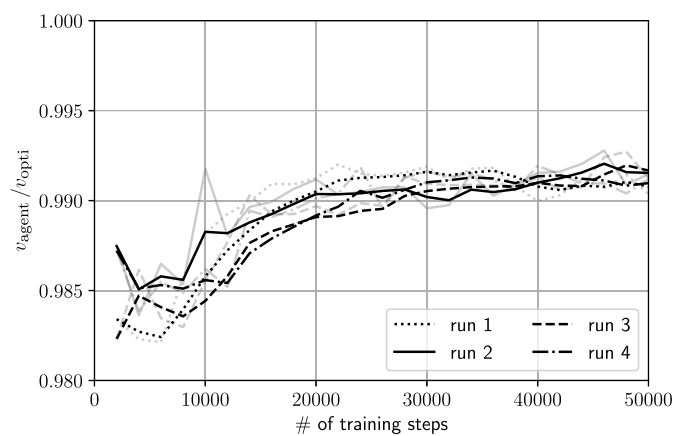


Fig. 9. Average of the final state value achieved by the agent on the test set during the training at Anytown-mod.

that the action-value estimator (the deep q -network) could not be developed further. However, the ability of the agent to solve the task cannot be judged by only the training rewards because if the reward is improperly specified, the optimum-looking behavior can be related to a completely false policy from the viewpoint of the original objective. Evolving false policies is a common problem in optimization tasks because most of them are exposed to the curse of Goodhart's law, see, for example, the work of Manheim and Garrabrant (2018) and Lehman et al. (2018).

Thus, two additional metrics are examined in the agent's performance. The most important aspect of the dueling deep q -network (DDQN) agent's behavior is whether it can reach comparable results to the reference solutions. This property was examined in the test scenarios by looking at the terminal state value of the episodes denoted by the reference state value of the same scenario. This is depicted in Fig. 9 for the Anytown-mod case. The value ratio rose above 99% in the first quarter of the training, and it stayed there from then on in three of the four training sessions. This implies that learning the optimal strategy in the sense of reaching a high state value is relatively easy in the Anytown-mod case. However, the value ratio does not tell about the number of steps required to reach the terminal state. Thus, the average length of the test scenarios is depicted in Fig. 10 for the Anytown-mod case. The average number

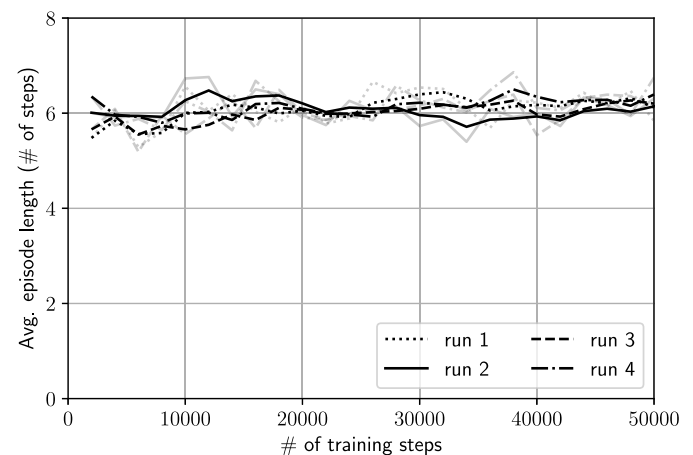


Fig. 10. Average length of an episode in the test set during the training at Anytown-mod.

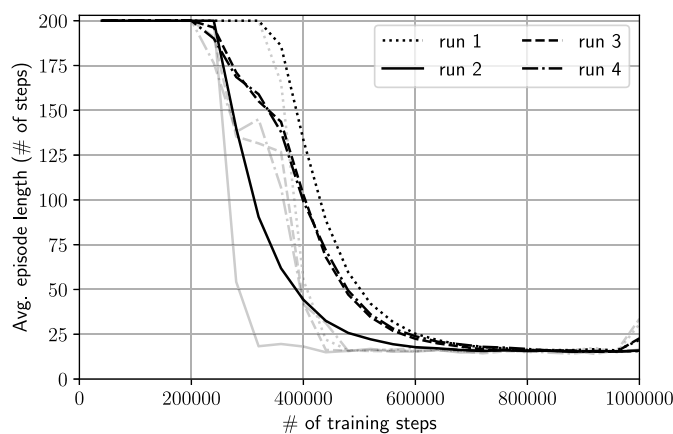


Fig. 11. Average length of an episode in the test set during the training at D-Town-mod.

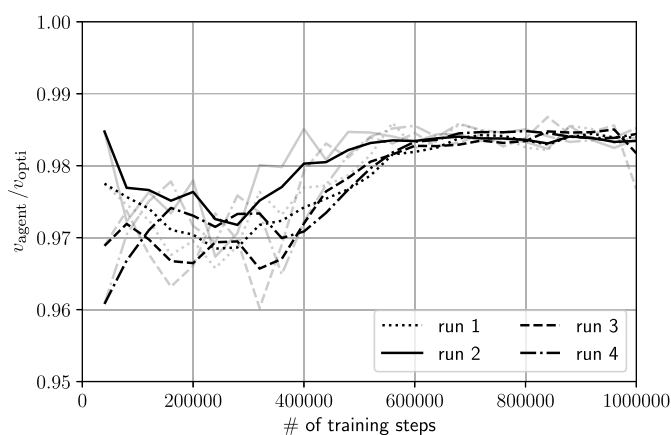


Fig. 12. Average of the final state value achieved by the agent on the test set during the training at D-Town-mod.

of steps taken before the termination was six, which means, that the agent took at most three active steps on average before terminating an episode with three consecutive siestas. As a reminder: there is only one pump station in this environment whose speed ratio can be varied between 0.7 and 1.1 in increments of 0.05. Moreover, the initial speed ratio was randomized uniformly between the feasibility limits; thus, a full sweep can be done in 8–11 steps with these settings (the initialization of the state is not counted as a step), and then the termination could be signaled with three additional siesta steps. Thus, three active steps in the Anytown-mod means that the agent started to move in the right direction. The fact that both the high rewards and the shortest trajectory were achieved at the beginning of the training means that this amount of training steps was unnecessarily high for the Anytown-mod problem.

The same metrics, that is, average value ratio and average episode length over the test scenarios during the training, are plotted from four different runs in Figs. 11 and 12 achieved with the D-Town-mod environment, respectively. According to Fig. 12, the DDQN also performed well in terms of value ratio in the more challenging environment. However, the absolute performance was slightly lower than in the previous case: the agent achieved at most 98% of the reference state value on average. The average episode length differed from the previous case, where the shortest trajectories were found soon after the beginning of the training. In this case,

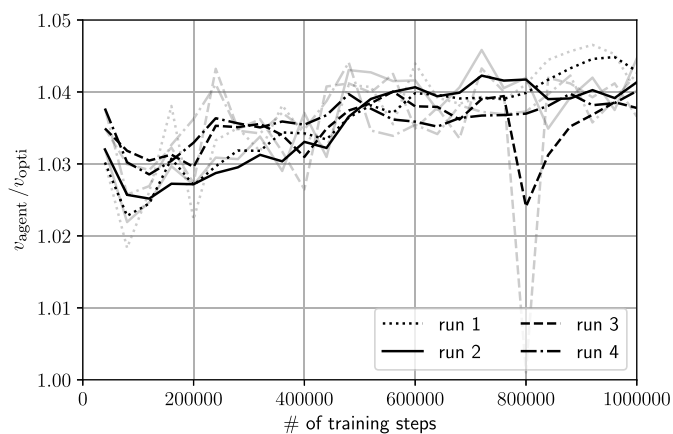


Fig. 13. Average of the final state value achieved by the agent on the test set during the training at D-Town-mod using one-shot random trial as a reference.

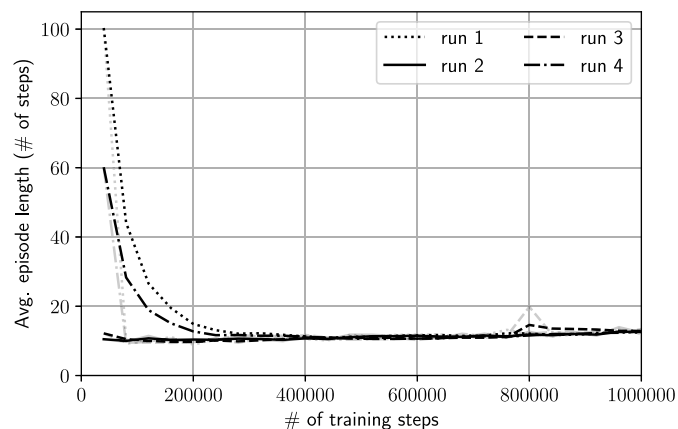


Fig. 14. Average length of an episode in the test set during the training at D-Town-mod using one-shot random trial as a reference.

Table 4. Agent performance on test scenarios

Run	Anytown-mod	D-Town-mod	D-Town-mod with one-shot RT
1	0.991	0.987	1.04
2	0.992	0.987	1.04
3	0.991	0.979	1.04
4	0.991	0.988	1.04

the agent exhausted all its possibilities to move around the parameter space until it learned that choosing a shorter trajectory leads to a higher reward.

Finally, Figs. 13 and 14 represent the same performance metrics for the D-Town-mod environment but with the reference solution acquired with one-shot random trial. So, in this case, the agent was guided by a suboptimal policy, but it can choose moves off policy thanks to the fairly high exploration factor at the beginning of the training. Owing to this, the DDQN agent performed reasonably well even with a poorly chosen reference algorithm because it outperforms that.

The average of the value ratios achieved in the test scenarios is summarized in Table 4. The performance on the test set was similar

to the performance on the validation set in the end of the training, which means the agent was able to generalize among different demand distributions.

Conclusions

A methodology is presented for instantaneous pump control in water distribution systems with reinforcement learning. The proposed technique is novel in the sense that it relies entirely on live measurement data in the decision-making process. This property makes real-time optimal control of the pumps possible in a smart water network.

The underlying algorithm of the agent is a dueling deep q -network. Because DDQN handles discrete action spaces only, the pump speed setting process was done by changing the speeds with fixed-size increments.

The performance of the agent was measured on two water networks with significantly different number of nodes, pipes, and pump stations. Metrics were defined as value ratio and episode length. The former measures the value of the terminal state compared to the reference found by the Nelder–Mead method, whereas the latter shows whether the agent used the shortest path to the optimum or was just randomly seeking over the parameter space. The novel approach was found to be useful because the agent can achieve comparable results to widespread optimization algorithms even with the restrictions mentioned previously. In the more challenging case, the agent got close to the reference solution with a difference in value ratio smaller than 2%. Because the agent got an extra bounty if it reached a value ratio higher than 98%, this performance ought to be upgraded by fine-tuning the rewarding function. Another critical aspect of the agent-based pump control is whether the agent can do its task in a limited number of time steps. The proposed rewarding algorithm was found to be appropriate to incite the agent to use the shortest possible trajectories towards the optimal state. In other words, the agent was able to solve the more challenging task in around 20 timesteps (n_{DDQN}) compared to the 70 evaluations ($3.5n_{\text{DDQN}}$) of the Nelder–Mead method and the few hundred evaluations ($\approx 10n_{\text{DDQN}}$) of the evolutionary methods. Besides the number of evaluations, the classic methods rely on the hydraulic model of the water network, but the proposed technique is able to rely only on measurement data after training.

Moreover, the exploratory behavior during the training provides an opportunity to outperform the reference algorithm that is used for the training guide. This phenomenon was also observed with the presented technique in the case when suboptimal guidance was used.

The proposed technique is a novel operation point optimizer for variable-speed pumps in the presented state that depends only on measurement data during control. It is also suitable for hybrid pump scheduling methods to assign optimal set-points to VSPs in real time. Because the deep q -network can be changed to handle temporal processes, the presented technique can be further developed to a real-time pump scheduling method for water distribution networks.

Data Availability Statement

Some or all data, models, or code generated or used during the study are available in a repository online in accordance with funder data retention policies [Reinforcement learning algorithms implementation (Hill et al. 2018), Python API for EPANET (Heinsbroek

2016), original numerical models of Anytown and D-Town (University of Exeter 1986), the code repository of the presented research, including everything to reproduce the results (Hajgató et al. 2020).]

Acknowledgments

The research presented in this paper has been supported by the BME-Artificial Intelligence FIKP Grant of Ministry of Human Resources (BME FIKP-MI/SC) and by the National Research, Development and Innovation Fund (TUDFO/51757/2019-ITM), Thematic Excellence Program. Bálint Gyires-Tóth is grateful for the financial support of the Doctoral Research Scholarship of Ministry of Human Resources (ÚNKP-19-4-BME-189) in the scope of New National Excellence Program and of János Bolyai Research Scholarship of the Hungarian Academy of Sciences.

References

- Abdallah, M., and Z. Kapelan. 2017. "Iterative extended lexicographic goal programming method for fast and optimal pump scheduling in water distribution networks." *J. Water Resour. Plann. Manage.* 143 (11): 04017066. [https://doi.org/10.1061/\(ASCE\)WR.1943-5452.0000843](https://doi.org/10.1061/(ASCE)WR.1943-5452.0000843).
- Abdallah, M., and Z. Kapelan. 2019. "Fast pump scheduling method for optimum energy cost and water quality in water distribution networks with fixed and variable speed pumps." *J. Water Resour. Plann. Manage.* 145 (12): 04019055. [https://doi.org/10.1061/\(ASCE\)WR.1943-5452.0001123](https://doi.org/10.1061/(ASCE)WR.1943-5452.0001123).
- Akiba, T., S. Sano, T. Yanase, T. Ohta, and M. Koyama. 2019. "Optuna." In *Proc., 25th ACM SIGKDD Int. Conf. on Knowledge Discovery & Data Mining—KDD '19*. New York: ACM Press.
- Baltar, A. M., and D. G. Fontane. 2008. "Use of multiobjective particle swarm optimization in water resources management." *J. Water Resour. Plann. Manage.* 134 (3): 257–265. [https://doi.org/10.1061/\(ASCE\)0733-9496\(2008\)134:3\(257\)](https://doi.org/10.1061/(ASCE)0733-9496(2008)134:3(257)).
- Bene, J. G. 2013. "Pump schedule optimisation techniques for water distribution systems." Ph.D. thesis, Dept. of Process and Environmental Engineering, Univ. of Oulu Graduate School.
- Bergstra, J. S., R. Bardenet, Y. Bengio, and B. Kégl. 2011. "Algorithms for hyper-parameter optimization." In *Proc., Advances in Neural Information Processing Systems 24*, edited by J. Shawe-Taylor, R. S. Zemel, P. L. Bartlett, F. Pereira, and K. Q. Weinberger, 2546–2554. New York: Curran Associates.
- Bhattacharya, B., A. H. Lobrecht, and D. P. Solomatine. 2003. "Neural networks and reinforcement learning in control of water systems." *J. Water Resour. Plann. Manage.* 129 (6): 458–465. [https://doi.org/10.1061/\(ASCE\)0733-9496\(2003\)129:6\(458\)](https://doi.org/10.1061/(ASCE)0733-9496(2003)129:6(458)).
- Bottou, L., and O. Bousquet. 2008. "The tradeoffs of large scale learning." In *Proc., Advances in Neural Information Processing Systems 20 (NIPS 2007)*, edited by J. Platt, D. Koller, Y. Singer, and S. Roweis, 161–168. San Diego: Neural Information Processing Systems Foundation.
- Castelletti, A., F. Pianosi, and M. Restelli. 2013. "A multiobjective reinforcement learning approach to water resources systems operation: Pareto frontier approximation in a single run." *Water Resour. Res.* 49 (6): 3476–3486. <https://doi.org/10.1002/wrcr.20295>.
- Celi, C. F. L., P. L. Iglesias-Rey, and F. J. M. Solano. 2017. "Energy optimization of supplied flows from multiple pumping stations in water distributions networks." *Procedia Eng.* 186 (1): 93–100. <https://doi.org/10.1016/j.proeng.2017.03.214>.
- Cybenko, G. 1989. "Approximation by superpositions of a sigmoidal function." *Math. Control Signals Syst.* 2 (4): 303–314. <https://doi.org/10.1007/BF02551274>.
- Guo, G., S. Liu, Y. Wu, J. Li, R. Zhou, and X. Zhu. 2018. "Short-term water demand forecast based on deep learning method." *J. Water Resour. Plann. Manage.* 144 (12): 04018076. [https://doi.org/10.1061/\(ASCE\)WR.1943-5452.0000992](https://doi.org/10.1061/(ASCE)WR.1943-5452.0000992).

- Hajgat6, G., B. Gyires-T6th, and G. Pa6l. 2020. "RL-WDS." Accessed February 14, 2020. <https://github.com/BME-SmartLab/rl-wds>.
- Heinsbroek, A. 2016. "EPYNET." Accessed February 14, 2020. <https://github.com/Vitens/epynet>.
- Hill, A., et al. 2018. "Stable baselines." Accessed February 14, 2020. <https://github.com/hill-a/stable-baselines>.
- Lehman, J., et al. 2018. "The surprising creativity of digital evolution: A collection of anecdotes from the evolutionary computation and artificial life research communities." Preprint, submitted March 9, 2018. <https://arxiv.org/abs/1803.03453v3>.
- Machell, J., S. R. Mounce, and J. B. Boxall. 2010. "Online modelling of water distribution systems: A UK case study." *Drinking Water Eng. Sci.* 3 (1): 21–27. <https://doi.org/10.5194/dwes-3-21-2010>.
- Mahootchi, M., H. R. Tizhoosh, and K. Ponnambalam. 2007. "Reservoir operation optimization by reinforcement learning." *J. Water Manage. Model.* 15 (1): 165–184. <https://doi.org/10.14796/JWMM.R227-08>.
- Mala-Jetmarova, H., N. Sultanova, and D. Savic. 2018. "Lost in optimisation of water distribution systems? A literature review of system design." *Water* 10 (3): 307. <https://doi.org/10.3390/w10030307>.
- Manheim, D., and S. Garrabrant. 2018. "Categorizing variants of Goodhart's law." Preprint, submitted March 13, 2018. <https://arxiv.org/abs/1803.04585>.
- McClymont, K., E. Keedwell, D. Savić, and M. Randall-Smith. 2013. "A general multi-objective hyper-heuristic for water distribution network design with discolouration risk." *J. Hydroinf.* 15 (3): 700–716. <https://doi.org/10.2166/hydro.2012.022>.
- Mnih, V., K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. 2013. "Playing Atari with deep reinforcement learning." Preprint, submitted December 19, 2013. <https://arxiv.org/abs/1312.5602>.
- Nair, V., and G. E. Hinton. 2010. "Rectified linear units improve restricted Boltzmann machines." In *Proc., 27th Int. Conf. on Machine Learning, ICML'10*, 807–814. Brookline, MA: Microtome Publishing. <http://dl.acm.org/citation.cfm?id=3104322.3104425>.
- Nardo, A. D., et al. 2018. "On-line measuring sensors for smart water network monitoring." In *Proc., 13th Int. Conf. on Hydroinformatics*. Palermo, Italy: Univ. of Palermo.
- Odan, F. K., L. F. R. Reis, and Z. Kapelan. 2015. "Real-time multiobjective optimization of operation of water supply systems." *J. Water Resour. Plann. Manage.* 141 (9): 04015011. [https://doi.org/10.1061/\(ASCE\)WR.1943-5452.0000515](https://doi.org/10.1061/(ASCE)WR.1943-5452.0000515).
- Pascual, J., J. Romera, V. Puig, R. Creus, and M. Minoves. 2011. "Operational predictive optimal control of Barcelona water transport network." *IFAC Proc. Volumes* 44 (1): 10571–10578. <https://doi.org/10.3182/20110828-6-IT-1002.01079>.
- Razavi, S., B. A. Tolson, and D. H. Burn. 2012. "Review of surrogate modeling in water resources." *Water Resour. Res.* 48 (7): W07401. <https://doi.org/10.1029/2011WR011527>.
- Ritter, J., G. Corzo, D. P. Solomatine, and H. Angarita. 2020. "Multiobjective direct policy search using physically based operating rules in multireservoir systems." *J. Water Resour. Plann. Manage.* 146 (4): 05020002. [https://doi.org/10.1061/\(ASCE\)WR.1943-5452.0001159](https://doi.org/10.1061/(ASCE)WR.1943-5452.0001159).
- Rossmann, L. A. 1993. "EPANET: An advanced water quality modeling package for distribution systems." In *Proc., Annual Conf.—American Water Works Association*, 411–418. Washington, DC: USEPA.
- Savic, D., H. Mala-Jetmarova, and N. Sultanova. 2018. "History of optimization in water distribution system analysis." In Vol. 1 of *Proc., WDSA/CCWI Joint Conf.* Boca Raton, FL: CRC Press.
- Simpson, T. W., A. J. Booker, D. Ghosh, A. A. Giunta, P. N. Koch, and R.-J. Yang. 2004. "Approximation methods in multidisciplinary analysis and optimization: A panel discussion." *Struct. Multidiscip. Optim.* 27 (5): 302–313. <https://doi.org/10.1007/s00158-004-0389-9>.
- Sutton, R. S., and A. G. Barto. 2018. *Reinforcement learning*. Cambridge, MA: MIT Press.
- Taormina, R., and S. Galelli. 2018. "Deep-learning approach to the detection and localization of cyber-physical attacks on water distribution systems." *J. Water Resour. Plann. Manage.* 144 (10): 04018065. [https://doi.org/10.1061/\(ASCE\)WR.1943-5452.0000983](https://doi.org/10.1061/(ASCE)WR.1943-5452.0000983).
- United Nations. 2018. "World urbanization prospects: The 2018 revision." Accessed February 14, 2020. <https://population.un.org/wup/Publications/Files/WUP2018-Report.pdf>.
- United Nations. 2019. "World population prospects 2019: Data booklet." Accessed February 14, 2020. https://population.un.org/wpp/Publications/Files/WPP2019_DataBooklet.pdf.
- University of Exeter. 1986. "Centre for water systems." Accessed February 14, 2020. <http://emps.exeter.ac.uk/engineering/research/cws/>.
- Wang, Y., G. Cembrano, V. Puig, M. Urrea, J. Romera, D. Saporta, J. G. Valero, and J. Quevedo. 2017. "Optimal management of Barcelona water distribution network using non-linear model predictive control." *IFAC-PapersOnLine* 50 (1): 5380–5385. <https://doi.org/10.1016/j.ifacol.2017.08.1070>.
- Wang, Z., T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas. 2015. "Dueling network architectures for deep reinforcement learning." Preprint, submitted November 20, 2015. <https://arxiv.org/abs/1511.06581>.
- Zessler, U., and U. Shamir. 1989. "Optimal operation of water distribution systems." *J. Water Resour. Plann. Manage.* 115 (6): 735–752. [https://doi.org/10.1061/\(ASCE\)0733-9496\(1989\)115:6\(735\)](https://doi.org/10.1061/(ASCE)0733-9496(1989)115:6(735)).